

SYSTEM DESIGN

Complete Guide for Placement Interviews & DevOps

Beginner → Intermediate → Interview-Ready
20 Sections · Real-World Examples · Diagrams · Cheat Sheet

DNS	Load Balancing	Caching
CDN	Microservices	Kafka
CAP Theorem	Docker/K8s	Rate Limiting

Covers everything needed for SDE placements, DevOps roles & real-world engineering.

Table of Contents

- 1. Introduction to System Design** — What is System Design? · Evolution of Architecture
- 2. Basics of Web Architecture** — Client-Server Model · REST APIs · HTTP vs HTTPS
- 3. DNS - Domain Name System** — DNS Resolution Flow · DNS Hierarchy
- 4. Scaling Concepts** — Vertical vs Horizontal · Scale-in & Scale-out
- 5. Load Balancing** — Types · Algorithms · Diagram
- 6. Rate Limiting** — Token Bucket · Leaky Bucket · Sliding Window
- 7. Caching** — Strategies · Invalidation · CDN
- 8. CDN - Content Delivery Network** — How it Works · CloudFront
- 9. Queue & Messaging Systems** — Kafka · SQS · Pub/Sub · Fan-out
- 10. Database Scaling** — Sharding · Replication · CAP Theorem
- 11. Microservices Architecture** — Monolith vs Micro · Service Communication
- 12. High Throughput Systems** — Bottlenecks · Optimization
- 13. Cloud & Infrastructure** — EC2 · Auto Scaling · Lambda
- 14. Containerization & Orchestration** — Docker · Kubernetes
- 15. Event-Driven Architecture** — Fan-out Pattern · Event Sourcing
- 16. Networking Concepts** — VPC · Anycast · Latency
- 17. Reliability & Fault Tolerance** — Failover · Redundancy · Replication
- 18. Real System Design Examples** — URL Shortener · Chat System · Instagram Feed
- 19. Interview Tips** — How to Approach · Step-by-Step Framework
- 20. Cheat Sheet** — Quick Revision Reference

§1 Introduction to System Design

What it is, why it matters, and how it evolved

1.1 What is System Design?

System Design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements. Think of it like being an architect — you're designing the blueprint of a software building before any code is written.

Analogy: Designing a city. You decide where roads go, where hospitals are, how electricity flows — before any construction. System Design does the same for software.

1.2 Why is System Design Important?

- **Interviews:** FAANG / top companies ask system design as a core round
- **DevOps:** Understanding system flows helps you deploy, monitor & scale better
- **Real World:** Poor design = crashes, data loss, and unhappy users
- **Career Growth:** Senior engineers are judged by their system thinking

1.3 Evolution of Architecture

Architecture Evolution Timeline

MAINFRAME ERA (1950s-70s)

One giant computer, dumb terminals connect to it
Problem: Single point of failure, not scalable

CLIENT-SERVER ERA (1980s-90s)

Clients request -> Server responds
Problem: Server bottleneck as users grow

WEB ERA (2000s)

Millions of users, need load balancers, databases
Problem: Managing hundreds of servers manually

CLOUD / MICROSERVICES ERA (2010s-now)

Auto-scaling, containers, serverless
Pay for what you use, deploy in seconds

1.4 Program vs Application vs System

Term	What it is	Example
Program	Single executable that does one task	A Python script that sorts a list
Application	Multiple programs working together with a UI	WhatsApp — UI + backend + DB
System	Large-scale distributed infrastructure	Netflix — CDN + microservices + DBs

§2 Basics of Web Architecture

How the internet actually works under the hood

2.1 Client-Server Architecture

Every web interaction follows this pattern: a Client (your browser/app) sends a request, the Server processes it, and sends back a response.

Client-Server Request-Response Model

```
USER (Browser/App)
  |
  | HTTP Request: GET /home
  v
+-----+
|  SERVER  | <- Processes request
| (Backend) | <- Reads from Database
+-----+
  |
  | HTTP Response: 200 OK + HTML/JSON
  v
USER sees the page / data
```

2.2 APIs — Application Programming Interfaces

An API is like a waiter in a restaurant. You (client) tell the waiter (API) what you want, the waiter goes to the kitchen (server/database), gets it, and brings it back to you. You never directly access the kitchen.

REST API: A set of rules for how clients and servers communicate. Uses HTTP methods: GET (fetch), POST (create), PUT (update), DELETE (remove).

REST API Methods

Client Request	Server Action
GET /users/123	-> Fetch user 123
POST /users	-> Create new user
PUT /users/123	-> Update user 123
DELETE /users/123	-> Delete user 123

Response always includes:

- Status Code (200=OK, 404=Not Found, 500=Error)
- Body (JSON data)
- Headers (Content-Type, Auth tokens)

2.3 HTTP vs HTTPS

Feature	HTTP	HTTPS
Encryption	No encryption	TLS/SSL encrypted

Security	Data visible to anyone	Data encrypted in transit
Port	80	443
Used for	Internal services (rare)	All public websites
Certificate	Not required	SSL Certificate required

Always use HTTPS in production. HTTP sends passwords and data as plain text — anyone on the same network (like a cafe WiFi) can intercept it.

§3 DNS – Domain Name System

The phonebook of the internet

3.1 What is DNS?

DNS translates human-readable domain names (like google.com) into IP addresses (like 142.250.80.46) that computers use to communicate. Without DNS, you'd have to memorize IP addresses for every website!

Analogy: DNS is like your phone's contact list. You search "Mom" and your phone finds her number (IP). You don't memorize numbers — DNS handles that for the internet.

3.2 DNS Resolution — Step by Step

DNS Resolution Flow

You type: www.example.com

```
graph TD
    A[You type: www.example.com] --> B[1. LOCAL CACHE (your browser/OS)]
    B --> C[2. RECURSIVE RESOLVER (your ISP's DNS)]
    C --> D[3. ROOT NAME SERVER (knows TLD servers)]
    D --> E[4. TLD SERVER (.com, .org, .in)]
    E --> F[5. AUTHORITATIVE DNS SERVER]
    F --> G[Your browser connects to 93.184.216.34  
Result cached for future requests (TTL)]
```

1. LOCAL CACHE (your browser/OS) -> Not found?

2. RECURSIVE RESOLVER (your ISP's DNS) -> Doesn't know? Ask Root

3. ROOT NAME SERVER (knows TLD servers) -> Returns address of .com TLD server

4. TLD SERVER (.com, .org, .in) -> Returns address of example.com's server

5. AUTHORITATIVE DNS SERVER -> Returns: 93.184.216.34

Your browser connects to 93.184.216.34
Result cached for future requests (TTL)

3.3 DNS Record Types

Record	Purpose	Example
A	Domain -> IPv4 Address	example.com -> 93.184.216.34
AAAA	Domain -> IPv6 Address	example.com -> 2001:db8::1
CNAME	Alias to another domain	www -> example.com
MX	Mail server for domain	example.com mail -> mail.example.com
NS	Authoritative name servers	example.com NS -> ns1.example.com
TTL	Time to cache the record	300 = cache for 5 minutes

TTL (Time To Live) is crucial — a low TTL means DNS changes propagate fast but increases DNS lookup load. A high TTL reduces load but slows propagation of changes.

§4 Scaling Concepts

How to handle growth from 100 to 100 million users

4.1 Vertical Scaling (Scale Up)

Add more power to the SAME machine — more CPU, RAM, or faster disk.

Vertical Scaling

```
BEFORE                                AFTER
+-----+                             +-----+
| Server |                             | BIGGER Server |
| 4 CPU  | ==>                       | 32 CPU      |
| 8 GB RAM|                             | 256 GB RAM  |
+-----+                             +-----+
```

+ Simple, no code changes + Good for databases (single consistent DB)
- Hardware limits (can't scale infinitely) - Expensive - Single point of failure

4.2 Horizontal Scaling (Scale Out)

Add MORE machines. Instead of one big server, run many smaller ones behind a load balancer.

Horizontal Scaling

```
BEFORE                                AFTER
+-----+                             +-----+ +-----+
| 1 Server | ==>                     |Server 1| |Server 2|
|           |                             +-----+ +-----+
+-----+                             +-----+ +-----+
                                   |Server 3| |Server 4|
                                   +-----+ +-----+
                                   ^
                                   Load Balancer distributes traffic
```

+ Unlimited scaling potential + No single point of failure
- Complex: needs load balancing, session management

4.3 Scale-in vs Scale-out

Action	Meaning	When Used
Scale Out	Add more servers	Traffic spike (Black Friday)
Scale In	Remove servers	Traffic drops (night time)
Scale Up	Bigger hardware on same machine	DB that needs more memory
Scale Down	Reduce hardware specs	Cost optimization

Cloud platforms (AWS, GCP) support **AUTO SCALING** — they automatically add or remove servers based on CPU/traffic metrics. You pay only for what you use.

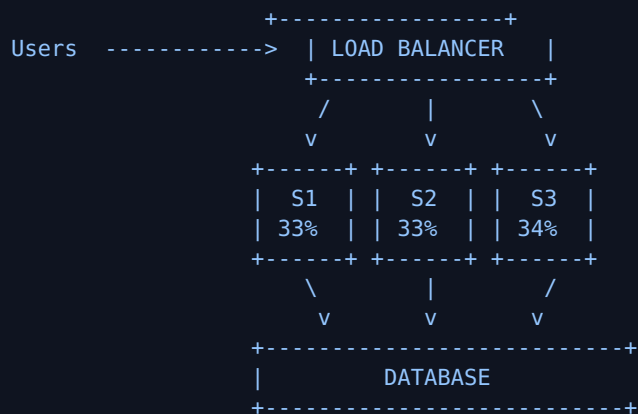
§5 Load Balancing

Distributing traffic so no single server gets overwhelmed

5.1 What is a Load Balancer?

A Load Balancer sits in front of your servers and routes incoming requests across multiple servers. Think of it like a traffic police officer directing cars to different lanes so no single lane jams up.

Load Balancer Architecture



5.2 Types of Load Balancers

Type	Works At	Routing Based On	Best For
Application LB (L7)	HTTP/HTTPS	URL path, headers, cookies	Web apps, microservices
Network LB (L4)	TCP/UDP	IP address, port	High-performance, gaming
DNS LB	DNS level	Geographic location	Global distribution

5.3 Load Balancing Algorithms

Load Balancing Algorithms

ROUND ROBIN

Request 1 -> Server 1
Request 2 -> Server 2
Request 3 -> Server 3
Request 4 -> Server 1 (wraps around)
+ Simple - Ignores server load

LEAST CONNECTIONS

Always routes to server with FEWEST active connections
Server 1: 10 connections
Server 2: 3 connections <- Next request goes here
Server 3: 7 connections
+ Smarter + Good for variable-length requests

IP HASH (Sticky Sessions)

Same user IP -> Always same server
+ Session persistence - Uneven distribution possible

Health Checks: Load balancers ping each server every few seconds. If a server doesn't respond, it's automatically removed from rotation until it recovers.

§6 Rate Limiting

Preventing abuse and protecting your system from being overwhelmed

6.1 What is Rate Limiting?

Rate limiting controls how many requests a user/IP can make in a given time window. It protects your API from abuse, DDoS attacks, and accidental overuse.

Real Example: Twitter API allows 300 tweets per 3 hours per user. If you exceed this, you get HTTP 429 (Too Many Requests). Login systems allow 5 failed attempts before a logout.

6.2 Token Bucket Algorithm

Imagine a bucket that fills with tokens at a fixed rate. Each request consumes one token. If the bucket is empty, the request is denied.

Token Bucket

Bucket capacity: 10 tokens Refill rate: 2 tokens/second

```
Second 0: [#####] 10 tokens
3 requests come in -> [##### ] 7 tokens left
Second 1: +2 tokens -> [##### ] 9 tokens
12 requests burst! -> 9 allowed, 3 REJECTED
```

- + Allows short bursts above average rate
- + Simple to implement
- Used by: AWS API Gateway, Stripe

6.3 Leaky Bucket Algorithm

Requests enter a bucket (queue) and leak out at a FIXED constant rate. Excess requests are dropped.

Leaky Bucket

```
Incoming: ||||| (burst of requests)
           v
+-----+
| o o o o o o | <- Queue (max 10)
+-----+      Excess dropped!
           v constant rate (1 req/sec)
Processed: o o o o o o o o o (smooth output)
```

- + Smooths out traffic bursts
- + Protects downstream services
- Burst requests are queued/dropped, no burst allowance

6.4 Sliding Window Algorithm

Sliding Window

Limit: 5 requests per 60 seconds

Fixed Window Problem:

[0-60s]: 5 requests used up at t=59

[60-120s]: 5 new requests at t=61

= 10 requests in 2 seconds! (window boundary exploit)

Sliding Window Fix:

At any point in time, look back 60 seconds

Count requests in that moving 60s window

Always enforces the true limit

+ Most accurate

+ No boundary exploits

- More memory (stores timestamps)

Algorithm	Burst Allowed?	Memory	Accuracy	Best Use Case
Token Bucket	Yes	Low	Good	APIs with burst traffic
Leaky Bucket	No	Low	Good	Smooth rate control
Sliding Window	Partial	High	Excellent	Strict rate enforcement

§7 Caching

Store results of expensive operations so you don't repeat them

7.1 What is Caching?

Caching stores frequently accessed data in fast-access memory (like RAM) so future requests can be served without hitting the slow database. It's one of the most powerful performance techniques in system design.

Analogy: Instead of going to the library every time you need a book, you keep the most-used books on your desk. Much faster! The desk = cache, library = database.

Cache vs No Cache

Request: "Get user profile for user_id=123"

WITHOUT CACHE:

Request -> Server -> DB -> Return

Time: ~100ms

WITH CACHE:

Request -> Server -> Cache HIT?

| YES

Return (1ms)

| NO

DB -> Cache -> Return
(~100ms, then cached)

Cache Hit Ratio = Hits / (Hits + Misses)

Goal: >90% hit rate for frequently accessed data

7.2 Cache Write Strategies

Strategy	How it Works	Pros	Cons
Write-Through	Write to cache AND DB simultaneously	Always consistent	Slower writes
Write-Back (Write-Behind)	Write to cache first, sync to DB later	Fast writes	Risk of data loss on crash
Write-Around	Write directly to DB, bypass cache	Cache not polluted	First read is always slow

7.3 Cache Eviction Policies

- **LRU** (Least Recently Used) — remove the item that hasn't been accessed in the longest time
- **LFU** (Least Frequently Used) — remove the item accessed fewest times
- **FIFO** (First In First Out) — remove the oldest item in cache
- **TTL** (Time To Live) — each item expires after a set time (e.g., 5 minutes)

Cache Stampede Problem: When cache expires, thousands of requests hit the DB simultaneously. Fix: use mutex locks or probabilistic early expiration to pre-warm the cache before it expires.

7.4 Where to Cache?

Level	Location	Speed	Example
L1 Cache	Inside CPU	< 1ns	CPU registers
Application Cache	Server RAM	< 1ms	Redis, Memcached
Distributed Cache	Shared across servers	~1-5ms	Redis Cluster
CDN Cache	Edge servers worldwide	~10-50ms	CloudFront, Fastly
Database Cache	Query result cache	~10ms	MySQL Query Cache

§8 CDN – Content Delivery Network

Serving content from servers closest to the user

8.1 What is a CDN?

A CDN is a globally distributed network of servers (called edge servers or PoPs — Points of Presence) that cache and serve static content (images, videos, CSS, JS) from locations geographically close to users.

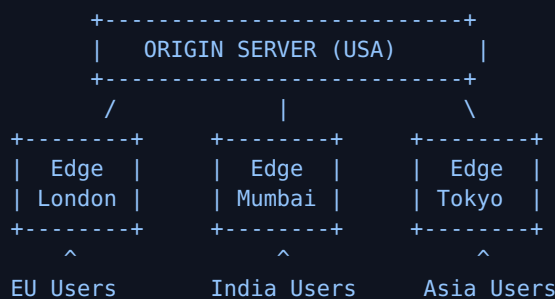
CDN Architecture

WITHOUT CDN:

User in Mumbai -> Server in USA (200ms latency!)

WITH CDN:

User in Mumbai -> CDN Edge in Mumbai (5ms!)
^ cached copy stored here



8.2 What Does a CDN Cache?

- **Static files:** Images, CSS, JavaScript, fonts
- **Videos:** Netflix uses CDN to stream video from the nearest edge
- **HTML pages:** For mostly-static sites
- **API responses:** For cacheable GET requests

Feature	Without CDN	With CDN
Latency (India user, US origin)	~200ms	~5-20ms
Origin Server Load	100% of requests	~5% (cache misses only)
Availability	Down if origin fails	CDN serves cached content
Cost at Scale	High bandwidth costs	Lower (CDN absorbs traffic)

Popular CDNs: AWS CloudFront, Cloudflare (most popular), Fastly, Akamai. CloudFront has 400+ edge locations, Cloudflare has 300+ cities. For interviews, remember: CDN = static content + low latency + offload origin.

§9 Queue & Messaging Systems

Decoupling services and handling async workloads

9.1 Why Do We Need Message Queues?

Imagine an e-commerce site on Black Friday. Orders come in faster than the payment system can process them. Instead of crashing, orders go into a queue. The payment service processes them at its own pace — this is asynchronous processing.

Message Queue Pattern

WITHOUT QUEUE (Synchronous):

```
User -> Order Service -> Payment Service -> Email -> Return
Problem: If Payment is slow, User waits. If Email fails, order fails!
```

WITH QUEUE (Asynchronous):

```
User -> Order Service -> QUEUE -> Return (instant!)
                        |
                        +--> Payment Consumer (reads queue)
                        +--> Email Consumer (reads queue)
                        +--> Analytics Consumer (reads queue)
```

Benefits: Faster response, fault tolerance, scalable consumers

9.2 Pub/Sub Model

Publisher/Subscriber: Publishers send messages to a TOPIC. Multiple subscribers independently receive and process all messages. Publishers don't know about subscribers.

Pub/Sub Fan-out Pattern

```
PUBLISHER          TOPIC          SUBSCRIBERS
(Order Service) --> [order.placed] --> Inventory Service
                                         --> Email Service
                                         --> Analytics Service
                                         --> Shipping Service
```

All 4 services get the SAME message independently.
This is called FAN-OUT: one message -> many consumers.

9.3 Apache Kafka vs AWS SQS

Feature	Apache Kafka	AWS SQS
Type	Distributed event streaming	Managed message queue
Message Retention	Days/weeks (replay possible)	14 days max
Throughput	Millions of msgs/sec	Thousands of msgs/sec
Replay	Yes — replay old events	No
Consumer Groups	Multiple groups, each gets all msgs	Message consumed once

Best For	Event sourcing, real-time analytics	Simple task queues
Used By	LinkedIn (creator), Netflix, Uber	AWS native applications

Key Interview Point: Use Kafka when you need to replay events, have multiple consumer groups, or need very high throughput (log aggregation, real-time analytics). Use SQS for simple task queues in AWS environments.

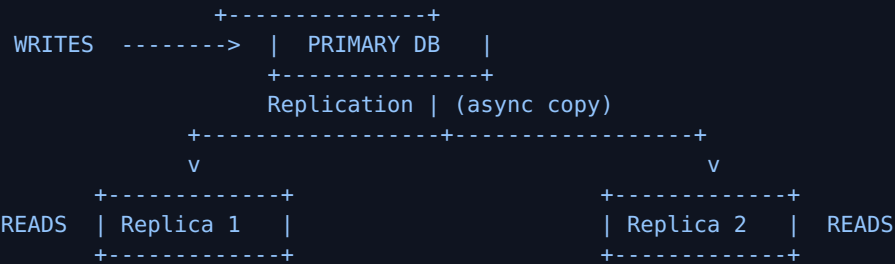
\$10 Database Scaling

Handling massive data and traffic — the hardest part of scaling

10.1 Read Replicas

In most applications, reads (SELECT) far outnumber writes (INSERT/UPDATE). We can have one primary DB for writes and multiple read replicas to handle read traffic.

Read Replicas



- + Scales read traffic easily
- Replication lag: replica may be slightly behind primary

10.2 Database Sharding

Sharding splits your database horizontally across multiple machines. Each shard holds a SUBSET of the data.

Database Sharding

User Table (100M rows) -> Split into 4 shards:

Shard 1: users with ID 1	- 25,000,000
Shard 2: users with ID 25M+1	- 50,000,000
Shard 3: users with ID 50M+1	- 75,000,000
Shard 4: users with ID 75M+1	- 100,000,000

Shard Key: the field used to split data (user_id, geo, etc.)

- + Unlimited horizontal scaling
- Cross-shard queries are complex
- Rebalancing shards when adding a new shard is painful
- No joins across shards easily

10.3 CAP Theorem Must Know

CAP Theorem states: in a distributed system, you can only GUARANTEE 2 out of 3: Consistency, Availability, or Partition Tolerance.

CAP Theorem

C = Consistency	Every read gets the most recent write
A = Availability	Every request gets a response (no timeout)
P = Partition Tol.	System works even if nodes can't communicate

REALITY: Network partitions ALWAYS happen in distributed systems.
So P is almost always required. You choose C or A.

CP Systems (Consistent + Partition Tolerant):

- > Bank databases, HBase, Zookeeper
- > During partition: returns error rather than stale data

AP Systems (Available + Partition Tolerant):

- > DNS, Cassandra, DynamoDB, CouchDB
- > During partition: returns potentially stale data

CA Systems: only possible in single-node (no network partitions!)

- > Traditional RDBMS (MySQL, PostgreSQL) in single-node mode

10.4 Strong vs Eventual Consistency

Type	Guarantee	Example	Trade-off
Strong Consistency	Read always returns latest write	Bank balance, stock prices	Higher latency, less availability
Eventual Consistency	Data converges to same value eventually	Social media likes, DNS	Lower latency, higher availability
Read-Your-Writes	You always see your own writes	User profile updates	Moderate

10.5 SQL vs NoSQL

Feature	SQL (Relational)	NoSQL
Data Structure	Tables with fixed schema	Documents, key-value, graphs
Scaling	Vertical (mainly)	Horizontal (naturally)
Consistency	Strong (ACID)	Eventual (BASE)
Use When	Complex queries, transactions	High volume, flexible schema
Examples	PostgreSQL, MySQL	MongoDB, Redis, Cassandra

§11 Microservices Architecture

Breaking a big application into small, independent services

11.1 Monolith vs Microservices

Monolith vs Microservices

MONOLITH

```
+-----+
| Auth | Orders | Payment |
| Products | Users | Search |
+-----+
```

One big codebase,
one deploy

MICROSERVICES

```
+-----+ +-----+ +-----+
| Auth | | Orders | | Payment|
| Svc  | | Svc   | | Svc   |
+-----+ +-----+ +-----+
+-----+ +-----+ +-----+
| Users | | Search | | Product|
| Svc   | | Svc   | | Svc   |
+-----+ +-----+ +-----+
Each is independent, own DB
```

11.2 Comparison Table

Aspect	Monolith	Microservices
Deployment	Deploy entire app each time	Deploy individual services
Scaling	Scale entire app	Scale only needed services
Tech Stack	Single language/framework	Each service can use different tech
Failure Impact	One bug = whole app down	One service fails, others continue
Complexity	Simple to start	Complex (networking, monitoring)
Best For	Small teams, startups, MVPs	Large teams, complex domains

11.3 Service Communication

Type	Protocol	When to Use	Example
Synchronous	REST/gRPC	Need immediate response	User service calls Auth service
Asynchronous	Kafka/SQS	No need to wait	Order placed -> notify inventory
Service Mesh	Istio/Envoy	Complex microservice networks	Netflix, Uber

API Gateway Pattern: Instead of clients calling each microservice directly, all requests go through an API Gateway (like AWS API Gateway). It handles auth, rate limiting, routing, and SSL termination in one place.

§12 High Throughput Systems

Handling millions of requests per second

12.1 Identifying Bottlenecks

Finding Bottlenecks

REQUEST FLOW – find where slowdowns happen:

```

DNS (1ms) -> Load Balancer (1ms) -> App Server (50ms?)
                                   |
                                   v
                        Database Query (200ms?) <-- BOTTLENECK
                                   |
                                   v
                        External API (500ms?) <-- BOTTLENECK

```

Common Bottlenecks:

1. N+1 DB queries (loop that makes 1 query per item)
2. Missing database indexes
3. Synchronous calls to slow external services
4. Large payload sizes
5. No caching on repeated expensive computations

12.2 Optimization Techniques

Problem	Solution	Impact
Slow DB queries	Add indexes, optimize query	10-100x speedup
Repeated computation	Cache results (Redis)	~99% reduction
Slow external APIs	Async calls + timeout + fallback	No blocking
Large DB table	Pagination, sharding	Constant query time
Image loading slow	CDN + compression + WebP	60-90% faster
High write load	Write-behind cache + batch writes	10x throughput

12.3 Key Metrics to Measure

- **Latency (p50/p95/p99):** 50th, 95th, 99th percentile response times — p99 matters most
- **Throughput:** Requests per second (RPS) your system handles
- **Error Rate:** % of requests that fail — aim for < 0.1%
- **Availability:** % uptime. 99.9% = 8.7 hrs/yr downtime. 99.99% = 52 mins/yr

§13 Cloud & Infrastructure

AWS basics every engineer should know

13.1 Cloud Computing Concepts

Model	What You Manage	Provider Manages	Example
IaaS	OS, Apps, Data	Hardware, network	AWS EC2
PaaS	Applications, Data	OS, runtime, infra	Heroku, Elastic Beanstalk
SaaS	Just using the app	Everything	Gmail, Dropbox, Slack
Serverless (FaaS)	Function code only	Server, OS, scaling	AWS Lambda

13.2 Key AWS Services

Key AWS Services Cheatsheet

COMPUTE:

- EC2 -> Virtual machine (rent a server)
- Lambda -> Run code without servers (pay per execution)
- ECS/EKS -> Run Docker containers / Kubernetes

NETWORKING:

- VPC -> Your private network in AWS
- ELB -> Elastic Load Balancer
- Route53 -> DNS service
- CloudFront -> CDN

STORAGE:

- S3 -> Object storage (files, images, backups)
- EBS -> Block storage (like a hard drive for EC2)
- RDS -> Managed relational DB (MySQL, PostgreSQL)
- DynamoDB -> Managed NoSQL DB
- ElastiCache -> Managed Redis/Memcached

MESSAGING:

- SQS -> Message queue
- SNS -> Pub/Sub notifications
- Kinesis -> Real-time streaming (like Kafka)

13.3 Auto Scaling

Auto Scaling

AUTO SCALING GROUP:

Min: 2 servers Desired: 4 servers Max: 20 servers

- CPU > 70% for 5 minutes? -> ADD 2 servers (scale out)
- CPU < 30% for 10 minutes? -> REMOVE 2 servers (scale in)

Launch Template: defines what new servers look like
(AMI, instance type, security groups, user-data script)

+ Handles traffic spikes automatically

§14 Containerization & Orchestration

Docker and Kubernetes — modern deployment essentials

14.1 What is Docker?

Docker packages your application and ALL its dependencies (OS libs, runtime, config) into a container. It runs identically on any machine. No more "works on my machine" problems!

Docker Explained

WITHOUT DOCKER:

Dev machine: Python 3.9
Prod server: Python 3.6
Colleague: Python 3.11
-> Dependency hell!

WITH DOCKER:

```
+-----+
|   CONTAINER   |
| App Code      |
| Python 3.9     |
| Dependencies   |
+-----+
Runs SAME on any machine
```

Dockerfile -> Build -> Docker Image -> Run -> Container
(recipe) (cook) (packaged app) (run) (running app)

14.2 What is Kubernetes (K8s)?

Kubernetes is a container orchestration system. When you have hundreds of containers, you need something to manage them — restarting failed ones, scaling, rolling updates, load balancing. That's Kubernetes.

Kubernetes Architecture

K8S CLUSTER:

```
+-----+
| CONTROL PLANE (Master) |
| API Server | Scheduler | Controller Manager |
+-----+
|
+-----+
v         v         v
+-----+ +-----+ +-----+
| Node 1 | | Node 2 | | Node 3 |
| Pod Pod| | Pod Pod| | Pod Pod|
+-----+ +-----+ +-----+
```

Pod = smallest unit = 1 or more containers
Node = a VM/physical machine running pods
Deployment = desired state ("run 3 replicas of my app")
Service = stable endpoint to reach pods (load balancing)

Concept	What it does
Pod	Smallest deployable unit, runs 1+ containers
Deployment	Manages replicas, rolling updates, rollbacks
Service	Stable network endpoint, load balances across pods

Ingress	External HTTP routing to services (like a reverse proxy)
ConfigMap/Secret	Store config and sensitive data separately from code
HPA	Horizontal Pod Autoscaler — auto-scales pods by CPU/memory

§15 Event-Driven Architecture

Systems that react to events rather than direct calls

15.1 What is Event-Driven Architecture?

Instead of services directly calling each other, they emit events. Other services subscribe and react independently. This creates loosely coupled, highly scalable systems.

Event-Driven vs Traditional

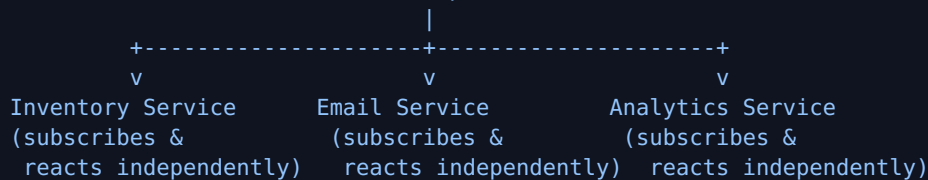
TRADITIONAL (Tight Coupling):

```
Order Service -> calls -> Inventory Service
                  -> calls -> Email Service
                  -> calls -> Analytics Service
```

Problem: If any downstream fails, order fails

EVENT-DRIVEN (Loose Coupling):

```
Order Service -> emits -> [order.completed event]
```



Order Service doesn't know or care about downstream services!

15.2 Instagram Feed Fan-Out Example

Feed Fan-out Pattern

User A (celebrity, 10M followers) posts a photo:

PUSH Model (Fan-out on Write):

```
Photo posted -> Write to feed of ALL 10M followers
+ Read is fast    - Write is expensive (10M writes!)
```

PULL Model (Fan-out on Read):

```
User opens feed -> Query all followees' posts -> Merge
+ Write is fast    - Read is slow (query 1000s of followees)
```

HYBRID (Instagram's approach):

```
Regular users -> Push model (fan-out on write)
Celebrity users -> Pull model at read time
Best of both worlds!
```

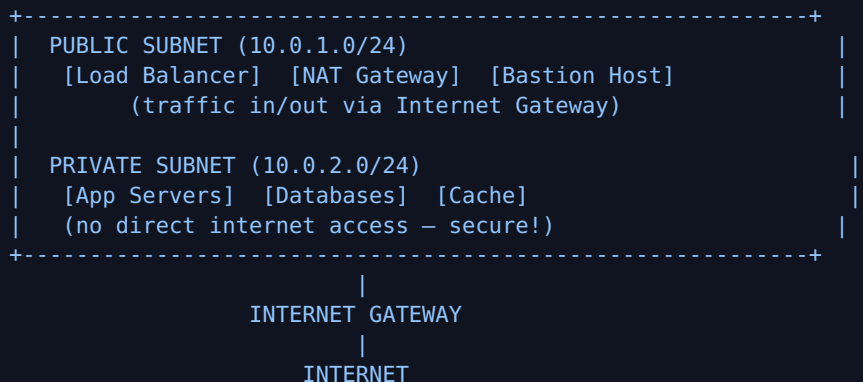
§16 Networking Concepts

VPC, latency, and network fundamentals

16.1 VPC — Virtual Private Cloud

VPC Architecture

YOUR VPC (10.0.0.0/16):



16.2 Latency Cheat Sheet

Operation	Typical Latency
L1 Cache access	< 1 nanosecond
RAM access	~100 nanoseconds
SSD read	~0.1 milliseconds
Same datacenter network	< 1 millisecond
Redis cache hit	~0.5 milliseconds
Database query (indexed)	~1-10 milliseconds
Same region cloud call	~1-5 milliseconds
CDN edge to user	~5-50 milliseconds
Cross-continent network	~100-300 milliseconds
HDD disk seek	~10 milliseconds

16.3 Anycast Routing

Anycast assigns the same IP address to multiple servers worldwide. When you request that IP, the network automatically routes you to the nearest server. Cloudflare and Google's 8.8.8.8 DNS use Anycast.

§17 Reliability & Fault Tolerance

Building systems that don't fail — and recover when they do

17.1 Failover

Failover Patterns

ACTIVE-PASSIVE FAILOVER:

Primary Server (active) <- All traffic
Secondary Server (passive) <- Standing by, syncing data
Primary fails -> Health check detects -> Traffic switches to Secondary
Failover time: ~30-60 seconds

ACTIVE-ACTIVE FAILOVER:

Server 1 (active) <--- Load Balancer ---> Server 2 (active)
Both handle traffic. If one fails, other gets 100%
Failover time: Instant (milliseconds)

Active-Active is more resilient but harder to manage

17.2 Redundancy & Replication

Concept	What it means	Example
Redundancy	Have duplicate components	Two load balancers, two DBs
Replication	Copy data to multiple nodes	DB replication across AZs
Multi-AZ	Deploy across availability zones	AWS: us-east-1a + us-east-1b
Multi-Region	Deploy across geographic regions	US + Europe + Asia
Circuit Breaker	Stop calling failing services	Return fallback instead of error
Retry with Backoff	Retry failed requests with delay	1s, 2s, 4s, 8s delays

SLA: Service Level Agreement. 99.9% uptime = 8.7 hours of downtime/year. 99.99% (four nines) = 52 minutes/year. Netflix and AWS target 99.99%+. Each additional "9" requires exponentially more engineering effort.

§18 Real System Design Examples

Step-by-step walkthroughs of common interview problems

18.1 URL Shortener (like bit.ly)

Design a service that converts long URLs into short URLs (bit.ly/abc123).

URL Shortener Design

REQUIREMENTS:

100M URLs shortened/day, 10:1 read:write ratio
Short URL must be unique, 7 characters, permanent

CREATE FLOW:

User: POST /shorten {url: "https://long.com/path"}
-> App Server generates hash (MD5/Base62 of URL + ID)
e.g. "abc1234" -> Store in DB: {abc1234 -> https://long.com}
-> Return: https://bit.ly/abc1234

REDIRECT FLOW:

User: GET /abc1234
-> Check Cache (Redis) -> HIT -> redirect 301
-> MISS -> DB lookup -> found -> cache it -> redirect 301

DB Schema: id | short_code | original_url | created_at | expiry
Index on: short_code (primary key lookup)

Component	Technology	Reason
API Server	Node.js / Go	High concurrency, fast I/O
Database	MySQL/PostgreSQL	Simple schema, ACID for unique codes
Cache	Redis	Read-heavy (90% reads), low latency
Load Balancer	AWS ALB	Distribute traffic across API servers
ID Generation	Snowflake ID / Auto-increment	Unique, sortable IDs

18.2 Chat System (like WhatsApp)

Design a real-time messaging system supporting 1:1 and group chats.

Chat System Design

KEY CHALLENGE: Real-time delivery (not polling!)

CONNECTION:

Client <----- WebSocket (persistent) -----> Chat Server

MESSAGE FLOW (User A sends to User B):

User A -> WebSocket -> Chat Server A
Chat Server A -> Message Queue (Kafka)
Chat Server B reads queue -> delivers to User B via WebSocket
Also: Message stored in DB for history

OFFLINE USERS:

User B offline? -> Store in DB, deliver when they reconnect
Push notification via FCM/APNs to wake up app

SCALE:

1B users, 100M daily active, 50B msgs/day
Need: Connection servers, Message brokers, DB sharding by chat_id
HBase/Cassandra: great for time-series message storage

18.3 Instagram Feed System

Instagram Feed Design

CORE FEATURES: Post photos, Follow users, See feed

PHOTO UPLOAD FLOW:

User -> API Server -> S3 (raw photo storage)
Async Job -> Resize/compress -> CDN (serve to users)

FEED GENERATION (Hybrid Fan-out):

Regular user posts -> Fan-out to followers' feed cache (Redis)
Celebrity posts -> NOT pre-fanned out (too expensive!)

FEED READ FLOW:

User opens app -> Fetch pre-computed feed from Redis
-> Merge with celebrity posts fetched live
-> Sort by timestamp/algorithm

DATA STORES:

Users, Follows -> MySQL (relational, critical)
Posts metadata -> Cassandra (write-heavy, scalable)
Photos -> S3 + CloudFront CDN
Feed cache -> Redis (pre-computed, sorted sets)
Search -> Elasticsearch

§19 Interview Tips & Framework

How to nail system design rounds consistently

19.1 The 6-Step Framework

6-Step Interview Framework

STEP 1 -- CLARIFY REQUIREMENTS (3-5 mins)

Ask: What features? Read/write ratio? Scale? Consistency needs?

Example: "Is this a global system? How many users per day?"

STEP 2 -- ESTIMATE SCALE (2-3 mins)

Calculate: Storage, bandwidth, QPS needed

1M users, 10 req/user/day = ~116 RPS average

STEP 3 -- HIGH-LEVEL DESIGN (5-7 mins)

Draw: Client -> LB -> Servers -> DB/Cache/Queue

Cover the happy path first, end-to-end

STEP 4 -- DEEP DIVE (10-15 mins)

Focus on 2-3 most critical components

DB schema, API design, bottlenecks

STEP 5 -- SCALING & BOTTLENECKS (5 mins)

"Where will this break at 10x scale?"

Introduce caching, sharding, CDN

STEP 6 -- WRAP UP

Summarize trade-offs made, mention what you'd improve

19.2 Common Mistakes to Avoid

Mistake	What to Do Instead
Jumping to solution without clarifying	Always ask 3-5 clarifying questions first
Over-engineering from the start	Start simple, then scale as needed
Ignoring trade-offs	Every choice has pros/cons — voice them
Going silent for too long	Think aloud, narrate your reasoning
Forgetting non-functional requirements	Address availability, consistency, latency
Picking tech without reason	"I'd use Kafka here because writes are high volume..."

19.3 Numbers Every Engineer Should Know

Metric	Approximate Value
1 million seconds	~11.5 days
1 billion seconds	~31.7 years

Characters in a tweet	~280 bytes
Average photo size	~3 MB
Average video (1 min, 720p)	~50 MB
1 TB	1,000 GB = 1 trillion bytes
Typical DB server throughput	~1,000-10,000 queries/sec
Redis throughput	~100,000 ops/sec

§20 System Design Cheat Sheet

Quick revision reference — everything on two pages

Key Concepts at a Glance

SCALING

Vertical = bigger machine | Horizontal = more machines. Scale-out for stateless services, scale-up for DBs.

LOAD BALANCER

Distributes traffic. L4=TCP, L7=HTTP. Algorithms: Round Robin, Least Connections. Always run health checks.

CACHING

Redis/Memcached. Strategies: Write-through (consistent), Write-back (fast). LRU eviction. Cache stampede fixed with a mutex.

CDN

Edge servers globally. Serves static files. CloudFront/Cloudflare. Reduces latency from ~200ms to ~5ms.

DATABASE

SQL = ACID, NoSQL = BASE. Read replicas for reads. Sharding for huge data. Index on query fields.

CAP THEOREM

Can't have C+A+P together. P is always needed. CP = banks, Zookeeper. AP = DNS, Cassandra, DynamoDB.

MESSAGE QUEUE

Async decoupling. Kafka = replay + high throughput. SQS = simple AWS queue. Pub/Sub = fan-out to multiple consumers.

RATE LIMITING

Token Bucket = bursts allowed. Leaky Bucket = smooth. Sliding Window = most accurate. HTTP 429 = Too Many Requests.

MICROSERVICES

Independent services, own DBs. API Gateway for routing. Service mesh for complex networks. Async via events.

CONTAINERS

Docker = package app. Kubernetes = orchestrate containers. Pod = unit, Deployment = desired state, HPA = autoscale.

DNS

Resolves domain -> IP. Flow: Cache -> Recursive -> Root -> TLD -> Authoritative. TTL controls cache duration.

RELIABILITY

Failover: Active-Passive (slow) vs Active-Active (instant). Multi-AZ for high availability. Circuit breaker prevents cascade failures.

Interview Decision Framework — When to Use What

Need low latency reads?	-> ADD CACHE (Redis)
Need to serve static files fast?	-> USE CDN
Lots of users, need to scale?	-> HORIZONTAL SCALE + LOAD BALANCER
Write-heavy workload?	-> WRITE-BACK CACHE + ASYNC QUEUE
Read-heavy DB?	-> READ REPLICAS
Huge DB table?	-> SHARD by natural key
Services calling each other?	-> DECOUPLE with message queue
Need real-time?	-> WEBSOCKETS (chat, live feeds)
Need consistency (bank)?	-> SQL + Strong consistency (CP)
Need scale + speed (social)?	-> NoSQL + Eventual consistency (AP)
Need auto-deploy?	-> DOCKER + KUBERNETES
Traffic spikes unpredictable?	-> SERVERLESS (Lambda) or AUTO SCALING

Availability Numbers

Availability	Downtime / Year	Downtime / Month	Target For
99% (two 9s)	87.6 hours	7.3 hours	Non-critical internal tools
99.9% (three 9s)	8.76 hours	43.8 minutes	Standard SaaS products
99.99% (four 9s)	52.6 minutes	4.4 minutes	Enterprise, AWS services
99.999% (five 9s)	5.26 minutes	26.3 seconds	Telecom, critical infrastructure

This guide covers everything needed for system design interviews at top tech companies and DevOps roles. Practice drawing diagrams by hand, time yourself to 45-minute design sessions, and always think about trade-offs. Good luck!